Inference

Lecture 4: what it costs to run a language model

Aarne Talman

10 November 2026

LDA-T318 | LLMs and Generative AI for NLP | University of Helsinki

Course overview

Wk	Dates	Tuesday	Thursday	Teacher
1	27 / 29 Oct	Language models	Pretraining	Aarne
2	3 / 5 Nov	Post-training	Lab 2: tune vs prompt	Aarne
3	**10 / 12 Nov**	**Inference**	**Lab 3: serve**	**Aarne**
4	17 / 19 Nov	Evaluation	Lab 4: evaluate	Jussi
5	24 / 26 Nov	RAG	Lab 5: RAG	Dmitry
6	1 / 3 Dec	Agents	Lab 6: agent	Dmitry
7	8 / 10 Dec	Course recap	Special topics (TBC)	All

Reusing a task and model across labs is recommended, not required.

There are no intermediate submissions; the final report and code archive determine the grade.

Lecture overview

1. Inference basics *15 min*
2. Decoding *10 min*
3. Prefill vs decode *15 min*
4. The KV cache *15 min*
5. Batching and the server *10 min*
6. Quantisation *10 min*
7. Speculative and constrained decoding *10 min*
8. Cost of serving *5 min*

Take away

By the end of today you should be able to explain where the time and memory go when a model answers, estimate how many users one GPU can serve, and compare two server settings fairly on speed and quality.

Inference basics

Inference: running a finished model, many times over

Training

Forward *and* backward passes over large batches; the parameters change at every step.
Compute-bound, run once per model version, and judged by loss per FLOP.

Inference

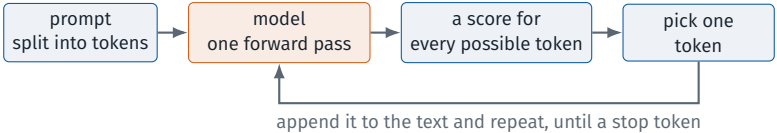
Forward passes only, with **frozen** parameters, for requests that arrive one by one, with different lengths, from many users at once. Judged by latency, throughput and cost per token, at a given quality.

A deployed model is trained once and queried millions of times, so inference usually dominates its lifetime compute. On the same GPU, training keeps the arithmetic units busy; inference often waits for memory.

Take away

Today's questions: where do the time and the memory go when a model answers, how many users can one GPU serve, and how can we make it cheaper without making the answers worse?

A MODEL WRITES ITS ANSWER ONE TOKEN AT A TIME



- The prompt is processed in **one** forward pass over all its positions in parallel; this produces the first output token.
- Every further token needs **its own** forward pass, conditioned on everything generated so far. A 500-token answer is 500 sequential passes, and they cannot be parallelised within one request.
- Each pass ends in logits over the vocabulary; a decoding rule turns them into one token (section 2).

The cost of **autoregressive generation** is therefore set by the number of *output* tokens much more than by the length of the prompt. The rest of the lecture explains why, and what servers do about it.

GPU MEMORY HOLDS THE WEIGHTS AND A GROWING KV CACHE

A server's GPU memory holds three things:

- **parameters**: number of parameters $\times$ bytes per parameter, e.g. 4B in bf16 (2 bytes) $=$ **8 GB**; FP8 halves it, INT4 quarters it;
- the **KV cache**: stored attention keys and values for every active request, growing with each token (section 4);
- **workspace**: activations of the current step, small at inference because nothing is kept for a backward pass.

Unlike training, there are no gradients or optimiser states, so the parameters are the fixed part and the KV cache is the part that decides how many requests fit.

The GPU

GH200 on Roihu: 96 GB of HBM, about 990 TFLOP/s in bf16, and about 4 TB/s memory bandwidth.

Whether a step is limited by **compute** or by **memory bandwidth** depends on how many FLOPs it does per byte it reads. For generation this is the central question of the lecture.

An inference engine serves many users through one API

An **inference engine** keeps the model resident on the GPU and schedules many concurrent requests onto it: it batches them, manages the KV cache, and streams tokens back. **vLLM** (used in Lab 3) and **SGLang** are the main open-source engines.

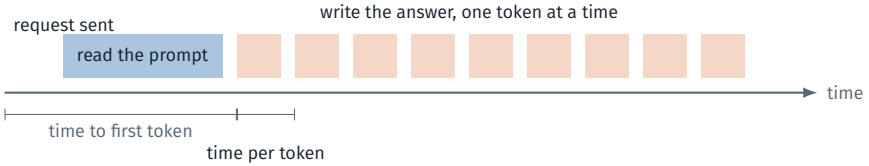
Most engines expose an OpenAI-compatible HTTP API, so the same client code, benchmark and scorer work with any of them. Decoding settings travel with each request, which is why they belong in the experiment record (section 2).

A request

```
POST /v1/chat/completions
{"model": "task",
 "messages": [{"role": "user",
               "content": "Is this review
     positive?..."}],
 "temperature": 0,
 "max_tokens": 32,
 "stream": true}
```

The answer arrives piece by piece (*streaming*), so the user can start reading before the model has finished.

What we measure: speed per user, total speed, quality

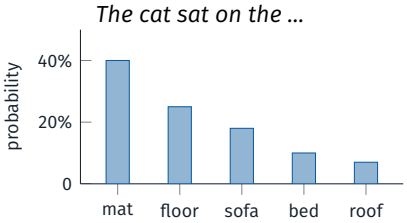


- **time to first token** (TTFT): how long until the answer starts
- **time per output token** (TPOT): how fast the answer then appears
- **throughput**: tokens per second for all users together, i.e. what the GPU produces
- **median** and **p95**: the typical request, and the slowest one in twenty
- **quality**: are the answers still as good? Always measured next to speed

Decoding

A DECODING RULE TURNS THE SCORES INTO THE NEXT TOKEN

Each forward pass gives a score for every token in the vocabulary. A softmax turns the scores into probabilities, and a **decoding rule** picks one.



- **Greedy** (temperature 0): always take the most likely token, *mat*. Same prompt, same answer.
- **Sampling**: draw at random by probability: *mat* 40% of the time, *floor* 25%, …
- **Temperature** and **top-*p*** shape the draw: low temperature favours likely tokens, high temperature gives unlikely ones a chance; top-*p* cuts off the unlikely tail.

Take away

These settings are not in the weights; they come with each request. The same model answers differently under different settings, so fix and record them. Lab 3 uses greedy.

Use greedy decoding unless something can pick the best sample

Task	Sensible default	Why
Classification, extraction, exact-match QA	temperature 0 (greedy)	there is one right answer, so variance only costs you
Code, maths, anything verified	temperature 0 for one shot; about 0.8 if you sample several and vote	diversity only pays when you have a verifier to pick the winner
Open generation, dialogue, summarisation	temperature 0.7, with top-p 0.9	greedy output is repetitive and flat, and a human is judging

Sample widely only when a human or an automatic checker can reject poor outputs. Otherwise sampling adds variance without a way to select the better result.

Source: Raschka, *Build Your Own Reasoning Model*, Oct 2026, on best-of-n and self-consistency.

Set decoding settings explicitly: the defaults differ between tools

The model, the library and the server each have their own default decoding settings, and they **do not agree**.

Same model, same prompt, no settings given

- Hugging Face `generate()`: **greedy**, the same answer every time
- vLLM server: **samples at temperature 1**, a different answer each time

(for Qwen3.5-0.8B; a model's `generation_config.json` can change both)

The rule

Set the decoding settings yourself in every request (`temperature`, `max_tokens`), and record them next to every number you report.

Lab 3 sets temperature 0 for both, so a difference in accuracy comes from the backend, not from the settings.

A comparison means nothing if the two runs used different settings.

Prefill vs decode

Prefill reads the prompt, decode writes the answer

Reading the prompt is **one big parallel step**; writing the answer is **many small steps, one after another**. Users feel them differently.



Prefill: read the prompt

All prompt tokens go through the model together, in one pass. Plenty of work for the GPU. A longer prompt means a longer wait for the first token.

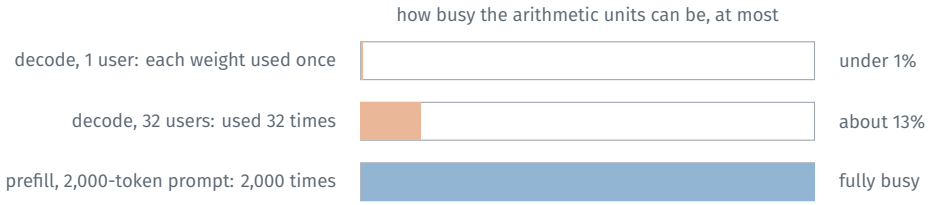
Decode: write the answer

One token per pass, and the next pass must wait for it. Each pass still reads all the weights, to produce a single token, so the GPU is mostly idle. A longer answer means more passes.

Most time usually goes to **decode**, which uses the GPU poorly: the rest of the lecture is about why, and what to do about it.

Why decode is slow: the GPU waits for memory

For every token, each weight must be **fetched** from memory and then **multiplied**. On a GH200, fetching is the slow part: while one weight arrives, the chip could do about 250 multiplications. So it is only busy if each fetched weight is used for **about 250 tokens**.



Prefill uses each fetched weight for every prompt token, so it keeps the GPU busy. Decode with one user uses it once, so the GPU mostly waits. Serving many users at once lets each fetch do more work: that is why **batching is the main fix** for decode.

Memory bandwidth sets the speed limit for one user

$$\text{tokens per second} \lesssim \frac{\text{memory bandwidth}}{\text{bytes of weights}}$$

That is: how many times per second can the card read the whole model?

Example: a 4B model in bf16 on one GH200

- 4 billion weights, 2 bytes each: **8 GB** to read for every single token
- the card moves **4.0 TB** of memory per second
- 8 GB goes into 4.0 TB about **500 times**, so 500 tokens a second, i.e. 2 ms each

That is the *upper limit* for one user. Real servers reach perhaps half of it.

Include the KV cache At 8k context, this example also reads about 1.2 GB of cached keys and values each step. The ceiling becomes

$$4.0\,\mathrm{TB/s} \div 9.2\,\mathrm{GB} \approx \textbf{430 tokens/s}.$$

Source: NVIDIA GH200 Grace Hopper Superchip datasheet (HBM3 and HBM3e variants); CSC, `docs.csc.fi/computing/systems-roihu`.

Most of the wait is decode, but users feel both delays

$$\text{total wait} = \underbrace{\text{TTFT}}_{\text{reading the prompt}} + \underbrace{\text{TPOT} \times (\text{tokens after the first})}_{\text{generating the answer}}$$

That is: you wait once for the prompt, then once per token produced.

Worked, 4B model, 2,000-token prompt, 500 output tokens:

- prefill: about 16 trillion operations, at the roughly 400 trillion per second the card really achieves: **40 ms**
- decode: 500 tokens at 5 ms each is **2.5 s**

Prefill is **1.6%** of the wall clock. Two thousand prompt tokens cost about as much time as **eight** generated ones.

Who feels what

TTFT — chat users, agents doing many short calls, anything with a spinner.

TPOT — anyone reading a streamed response. Anything under about 30 ms per token feels instant; 100 ms feels slow.

Neither — batch jobs. For offline scoring only *throughput* matters, so batch as much as memory allows.

The rest of the lecture: which technique helps which phase

Technique	helps prefill	helps decode
Bigger batches	no (already busy)	**yes, a lot**
INT4 weight-only quantisation	**can hurt**	**yes**
FP8 weights + activations	**yes**	**yes**
KV-cache quantisation	slightly	**yes**
Prefix caching	**yes, a lot**	no
Speculative decoding	no	**yes**

Take away

Work out which half of the request you are paying for before you optimise. For decode, bigger batches are the biggest lever, and what limits them is memory for each user's **KV cache**: the next section.

The KV cache

The KV cache trades memory for computation

At each decode step, the new position's query attends to the keys and values of *all* earlier positions, in every layer.

In a causal model those keys and values never change once computed, so the server computes them once and **stores** them: the **KV cache**.

Without the cache, step t would recompute t positions, which is quadratic work over a whole answer. With it, each step runs the layers for the new token only, and attention reads the cache.

The price

Memory that grows linearly with context length and with the number of concurrent requests, and that must be read back at every decode step.

Compute is traded for memory, which makes the cache, not the parameters, the usual limit on concurrency. The next slides size it.

The KV cache grows with every token: about 147 kB each

$$\text{KV bytes} = \text{requests} \times \text{tokens} \times \text{layers} \times \text{KV heads} \times \underbrace{\text{numbers per head}}_{128} \times \underbrace{2}_{\text{key and value}} \times \underbrace{2}_{\text{bytes each, bf16}}$$

That is: every layer stores two small vectors (a key and a value) for every token of every request. A typical 4B model (Qwen3-4B) has 36 layers and 8 KV heads, so

$$36 \times 8 \times 128 \times 2 \times 2 \;\approx\; 147\,\mathrm{kB} \text{ per token}$$

Why some models need less

Many models share one key and value between several attention *heads* (fewer "KV heads"), or look back only over a window of recent tokens. Both make the cache smaller.

Every decode step also *reads the whole cache*, so it costs speed as well as memory. Qwen3.5 keeps a KV cache in only one layer in four (the others use linear attention), so its cache grows about four times more slowly.

At long contexts, one user's cache outgrows the model

Context	KV / request	× weights (8 GB)	of 96 GiB
1 request @ 8k	1.2 GB	0.15×	1%
1 request @ 32k	4.8 GB	0.6×	5%
1 request @ 128k	19.3 GB	2.4×	19%

One user at 128k context occupies **more than twice the memory of the entire model**.

Double the context and you double the cache, which halves the number of users the GPU can hold.

Units, stated once

96 GiB = 103 GB.
A server typically reserves ≈ 90% of the card, and the weights take 8 GB, so

$$0.9 \times 103 - 8 \approx \mathbf{85}\,\mathrm{GB}$$

is what is actually available for KV. We use 85 GB from here on.

The KV cache, not the weights, limits how many users fit

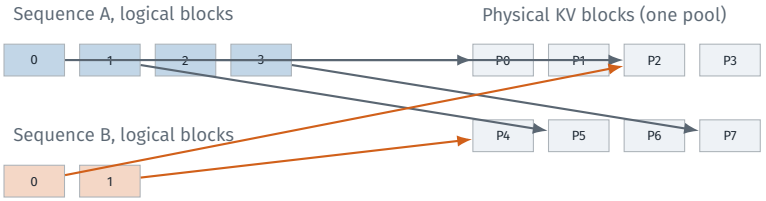
Requests at 8k	KV cache	of the 85 GB	
1	1.2 GB	1%	trivially fine
8	9.6 GB	11%	fine
32	38.5 GB	45%	fine
64	77 GB	91%	fine
70	84 GB	99%	at the limit
72	87 GB	102%	does not fit

At **128k** the same arithmetic gives **4** concurrent requests: sixteen times the context means sixteen times fewer users. The GPU could still compute more, but has no memory left.

The conclusion of this section

In this example, the KV cache sets the concurrency limit. The 4B model's weights take 8 GB, while its cache can grow to many times that at long contexts and high concurrency.

PagedAttention: cache memory in small blocks, almost no waste



- the cache is stored in small fixed-size **blocks**, handed out only when a request needs more
- a table per request records where its blocks are, so they need not be next to each other
- without this, servers reserved the maximum length for every request and wasted 60–80% of the cache memory; with it, **under 4%**
- requests that start with the same tokens can **share** blocks

Source: Kwon et al., SOSP 2023 (vLLM). SGLang keeps its cache in a similar shared pool.

Prefix caching: paying for the system prompt once

If a prompt **begins with** tokens whose KV blocks are still cached, the server reuses them and skips that part of prefill.

vLLM and SGLang do this automatically (SGLang calls it *RadixAttention*). The answers do not change; it only saves work, as long as the shared prefix is still in the cache.

In Lab 3 it is switched off for the speed test (`--no-enable-prefix-caching`), so repeated prompt starts do not make the server look faster than in normal use.

Worked: a 5-shot evaluation run

Shared prefix: system prompt + 5 examples ≈ 1,200 tokens.
Per-item suffix: ≈ 80 tokens.

Without prefix caching you prefill 1,280 tokens per item. With it, **80**.

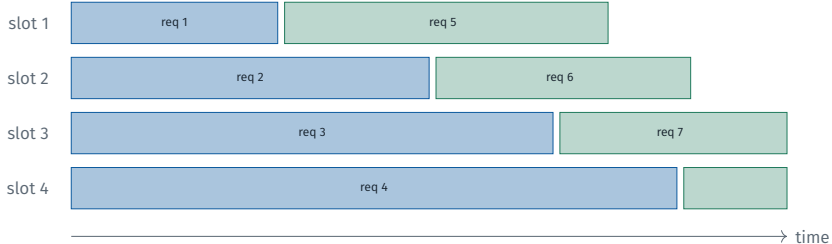
Over a 1,000-item test set that is up to 16× **less prefill work** while that prefix remains cached.

The same idea helps chat, agents and RAG, which repeat long instructions.

Source: Zheng et al., *SGLang: Efficient Execution of Structured Language Model Programs*, 2024 (RadixAttention); vLLM automatic prefix caching.

Batching and the server

Continuous batching: a free slot is refilled at every step



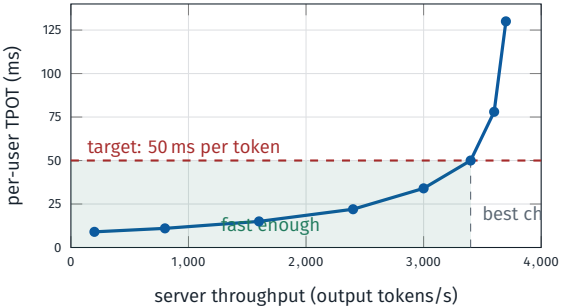
Older servers started a batch and waited for its slowest request before taking new ones. Now the server rebuilds the batch **at every step**:

- a request that has finished leaves at once and frees its memory
- a waiting request joins at the next step
- no slot sits idle while others finish

This gives roughly **2–4 times** more throughput at the same latency.

Source: Yu et al., OSDI 2022; Kwon et al., SOSP 2023.

Throughput and latency pull in opposite directions



More parallel requests give more tokens per second in total, but each user waits longer per token. Past the bend, extra requests only slow everyone down. Choose the point where the speed per user is still acceptable. In Lab 3 you can find this point for your own model.

Source: Shape after published vLLM and DistServe throughput–latency sweeps; values illustrative.

Four server flags set memory use and concurrency

```
# vLLM, as in Lab 3 (written on one line in practice)
vllm serve Qwen/Qwen3.5-0.8B
  --max-model-len 4096           # longest prompt + answer
  --max-num-seqs 64              # how many requests run at once
  --gpu-memory-utilization 0.8   # GPU memory for weights + KV cache
  --quantization fp8             # FP8 weights (leave out for bf16)
  --no-enable-prefix-caching     # prefix caching off: speed tests only
  --port 8000
```

`max-model-len`	the biggest lever on how many users fit; do not keep a 128k default if your task needs 2k
`max-num-seqs`	raising it past what the KV memory allows only makes requests wait inside the server
`gpu-memory-utilization`	how much of the GPU the server takes; lower it if you run out of memory
`enable-prefix-caching`	on by default; keep it on in real use

Quantisation

Quantisation: three separate places to use fewer bits

What	Size, 4B model	What lowering it buys
Weights	8 GB in bf16	fewer bytes read per decode step ⇒ directly faster decode
Activations (values inside the pass)	small, temporary	lets the GPU multiply in FP8 ⇒ faster *prefill*
KV cache	1.2 GB per 8k request	more concurrent users, and less to re-read every step

Fewer bits per number means fewer bytes to store and to read. These are **independent choices**. "The model is quantised" says little: always ask *which of the three* uses fewer bits, and in what format. An optional part of Lab 3 tests FP8 weights.

FP8 HALVES THE WEIGHTS AND SPEEDS UP BOTH PHASES

What it is

- weights (and often activations) use 8-bit numbers instead of 16-bit;
- each block of numbers also stores a scale factor, because 8 bits cover a much smaller range;
- the GH200 can multiply FP8 numbers directly, without unpacking them first.

Why it is used

- weights halve: 8 GB $\rightarrow$ 4 GB;
- less weight traffic speeds up decode;
- the GPU multiplies FP8 faster, which can also speed up prefill.

How to evaluate it Run the same test set and scorer, and report quality next to speed. Small average losses in papers do not guarantee that your language or task is unaffected.

Source: NVIDIA Hopper FP8 documentation; vLLM quantization documentation.

INT4 WEIGHTS (AWQ, GPTQ) SPEED UP DECODE ONLY

Store each weight in 4 bits and **unpack it to 16 bits just before use**. Only the stored weights shrink.

GPTQ rounds each layer's weights so that the layer's *output* changes as little as possible on sample data.

AWQ protects the small share of weights that matter most for the outputs.

Sources: Frantar et al., *GPTQ*, ICLR 2023; Lin et al., *AWQ*, MLSys 2024.

Why weight-only works at all

Decode waits for memory. Weights of 2 GB instead of 8 GB mean a quarter of the reading per token, so the upper limit goes from 500 to **2,000 tokens/s**. Prefill gains little, and the unpacking can even slow it down.

Both need **calibration data**: a few hundred example texts. Use data like your task; English web text is a poor choice for a Finnish model.

Quantisation damage hides in averages: score your own task

For FP8 and good INT4 the published quality loss is usually **under a point** on MMLU-style benchmarks. That average hides where the damage actually lands:

- **long-tail inputs**: rare formats, unusual prompts
- **non-English text**, especially morphologically rich languages
- **long reasoning chains**, where one bad token derails the rest
- **long contexts**, if the KV cache is quantised

So score it yourself

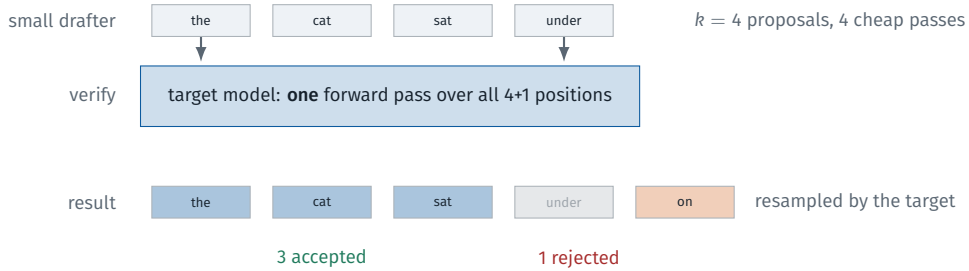
Serve the model in FP8, run the same test set and scorer, and put the score in the same table as the tokens per second (an optional part of Lab 3). **Always report the quality number next to the speed number.**

Take away

FP8 is a good default on a GH200; INT4 mainly speeds up decode. Either way, trust the quality number you measured on your own task.

Speculative and constrained decoding

Speculative decoding: a small model guesses, the big one checks



A small, fast model guesses the next few tokens; the big model checks all the guesses in **one** pass, which costs about the same as producing one token. Here one pass gives four tokens. The guesses are accepted in a way that keeps the output exactly as if the big model had produced it alone.

Sources: Leviathan et al., *Fast Inference from Transformers via Speculative Decoding*, ICML 2023; Chen et al., *Accelerating LLM Decoding with Speculative Sampling*, 2023.

Constrained decoding guarantees the format, not the answer

What it is

- Give the server a schema or grammar for the required output.
- At each step, mask tokens that would make the output invalid.
- Sample from the tokens that remain.

Why it is used

Applications often need parseable JSON, and retries add latency and cost.

What it guarantees

Valid syntax when generation completes successfully. The constraint removes invalid token sequences, but truncation and server failures still need handling.

What it does not guarantee

Anything about the content. The label inside valid JSON can still be wrong, and forcing the format can even make it worse: let the model reason freely and constrain only the final answer.

So report **both** the invalid-output rate and the task score. In vLLM you send a JSON schema with the request (`response_format`); an optional part of Lab 3 measures both.

Sources: Willard & Louf, *Efficient Guided Generation for LLMs*, 2023 (Outlines); Dong et al., *XGrammar*, 2024.

Cost of serving

Batching sets the cost per million tokens

$$\text{EUR per million tokens} = \frac{\text{EUR per GPU-hour}}{\text{tokens you got out of that hour}} \times 10^6$$

That is: you rent the GPU by the hour, so divide the rent by what the hour produced. In this example the rent is EUR 2.50/h and the GPU is busy only 40% of the time, so the result is divided by 0.4.

32 users at once, measured 2,500 tokens/s:

- $2500 \times 3600 = 9$ million tokens an hour
- EUR 2.50 for 9 million = EUR 0.28 per million
- busy only 40% of the time: **EUR 0.69 per million**

One user, measured 120 tokens/s: 0.43 million tokens an hour, the same arithmetic, **EUR 14.5 per million**.

The same GPU, the same model, 21× the price

Only the number of parallel requests changed. Batching decides most of the cost per token.

Public prices in late 2026: roughly \$0.02–0.10 per million output tokens for hosted 4B-class open models, \$5–15 for frontier ones.

Source: GPU price: representative on-demand H100/GH200 cloud rate, late 2026. Throughput: the kind of figure Lab 3 produces. API prices: public price lists, November 2026; they change quickly.

Three things to remember

1. **The two halves of a request hit different limits.** Reading the prompt (prefill) is limited by computation; writing the answer (decode) is limited by memory. Find out which one you are paying for before you optimise.

2. **The KV cache often sets the concurrency limit.** In the worked example the weights take 8 GB and the cache can take the rest of the GPU. Paging, prefix caching and quantisation all save cache memory.

3. **Measure latency and quality together.** Warm up the server, measure TTFT and TPOT separately, try several numbers of parallel requests, and report the median and p95. A faster setting is only useful if the answers stay good enough.

Lab 3: you run the same model with plain Hugging Face and with vLLM, benchmark and evaluate both, and compare the numbers with today's estimates.

A server configuration you can reuse

If the same endpoint suits your project, it can become **infrastructure for later labs**:

- **Week 5, RAG**: your server generates the answers from the retrieved text.
- **Week 6, agents**: the same server answers many short calls, so TTFT and prefix caching matter.
- **Week 7**: your demo runs against it.

The file worth keeping

Lab 3 keeps all server settings in `configs/model.json`, and `bash scripts/start_server.sh` starts the same server again in later weeks.

The steps are in `Week-3/Lab-3/README.md`.

A running server keeps its GPU memory whether or not anyone uses it. Stop it when you are done.

Reading

Serving systems

- Zheng et al. 2024, *SGLang* / RadixAttention
- Kwon et al. 2023, *PagedAttention* (vLLM)
- Yu et al. 2022, *Orca*, continuous batching
- Agrawal et al. 2024, *Sarathi-Serve*
- Zhong et al. 2024, *DistServe*
- Pope et al. 2023, *Efficiently Scaling Transformer Inference*

Kernels

- Dao et al. 2022, Dao 2023, *FlashAttention* 1 and 2
- Williams et al. 2009, the roofline model

Decoding

- Leviathan et al. 2023, speculative decoding
- Chen et al. 2023, speculative sampling
- Cai et al. 2024, *Medusa*; Li et al. 2024, *EAGLE*
- Willard & Louf 2023, *Outlines*; Dong et al. 2024, *XGrammar*

Quantisation

- Frantar et al. 2023, *GPTQ*; Lin et al. 2024, *AWQ*
- Hooper et al. 2024, *KVQuant*

Further reading

- Thinking Machines Lab 2025, *Defeating Nondeterminism in LLM Inference*